

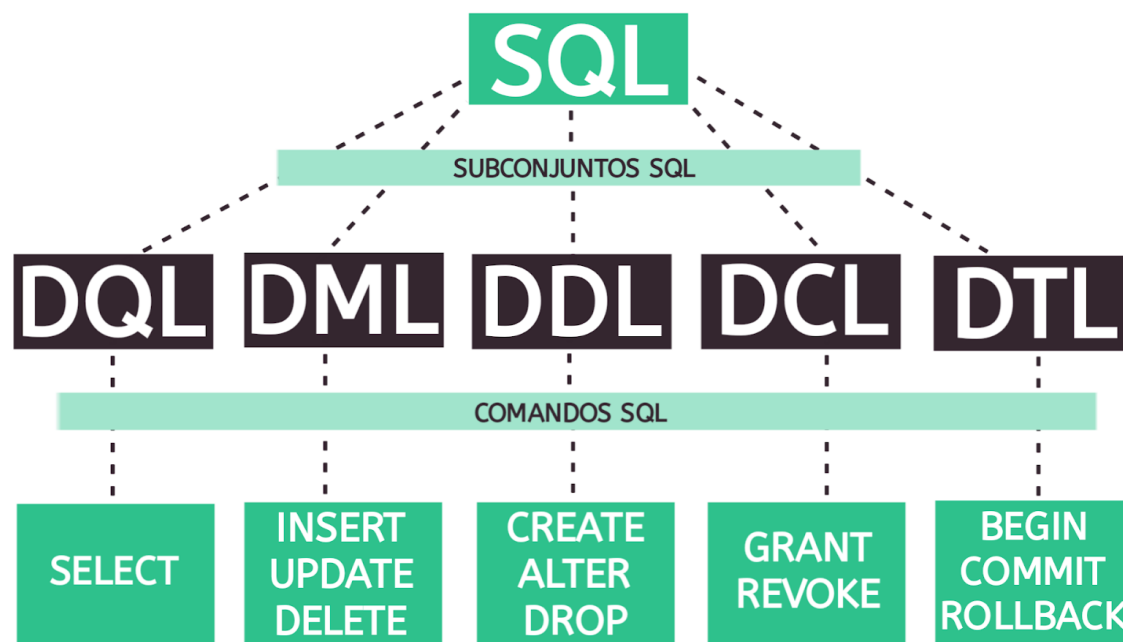
Introdução a Banco de Dados

Comandos SQL

Profa. Me. Eliza Maria da Cunha Bomfim

2026

Conheça os subconjuntos da linguagem SQL



DML: linguagem de manipulação de dados

O DML (Data Manipulation Language) é o subconjunto do SQL que **define os comandos usados para manipular os dados armazenados em um banco**. Esse é um dos conjuntos mais utilizados, pois ele fornece operadores que nos permitem inserir, excluir e alterar os registros de uma tabela, por exemplo. Os comandos mais importantes desse subconjunto são: INSERT, DELETE e UPDATE.

DQL: linguagem de consulta de dados

O DQL (Data Query Language) é o conjunto SQL que define o comando mais popular da linguagem, o SELECT. Esse comando é essencial para que possamos **consultar os dados que armazenamos em nosso banco**.

DDL: linguagem de definição de dados

O DDL (Data Definition Language) é o subconjunto SQL que apresenta **comandos usados para gerenciar as estruturas do banco de dados**. Com ele, podemos criar, atualizar e remover objetos da base, como tabelas e índices. Os comandos definidos pelo DDL são: CREATE, DROP e ALTER.

Instruções SQL

DDL

Criar novas tabelas

```
create table dept(  
  deptno  number(2,0) not null,  
  dname   varchar2(14),  
  loc     varchar2(13),  
  constraint pk_dept primary key (deptno)  
)
```

```
create table emp(  
  empno   number(4,0) not null,  
  ename   varchar2(10),  
  job     varchar2(9),  
  mgr     number(4,0),  
  hiredate date,  
  sal     number(7,2),  
  comm    number(7,2),  
  deptno  number(2,0),  
  constraint pk_emp primary key (empno),  
  constraint fk_deptno foreign key (deptno) references dept (deptno)  
)
```

Alterar tabela existente

```
ALTER TABLE emp  
ADD email  
varchar(255);
```

Remoção de tabela

```
DROP TABLE dept;
```

Not Null

```
ALTER TABLE emp MODIFY email  
varchar(255) NOT NULL;
```

Unique

```
ALTER TABLE emp ADD  
CONSTRAINT UC_Emp UNIQUE  
(empno,email);
```

Restrições (Constraints)

Chave Estrangeira (FK)

```
ALTER TABLE emp ADD CONSTRAINT fk_deptno  
FOREIGN KEY (deptno) REFERENCES  
dept(deptno);
```

Chave Primária (PK)

```
ALTER TABLE emp ADD CONSTRAINT  
PK_Emp PRIMARY KEY  
(empno,ename);
```

SQL INSERT

O comando INSERT é usado para **inserir novos registros em uma tabela do banco**. Agora que nossa tabela “funcionarios” já foi criada, podemos usar uma estrutura básica dessa instrução para começar a popular o banco. Observe:

```
INSERT INTO funcionarios (id, nome, cargo) VALUES (1, 'Fernando', 'Analista de Sistemas');
```

Nesse breve exemplo que mostramos, você deve reparar que “funcionarios” é o nome da tabela e “id”, “nome” e “cargo” são as colunas da tabela que estamos adicionando novos dados.

SQL UPDATE

Já o comando UPDATE é utilizado para **atualizar os registros que foram armazenados no banco**. Essa declaração é usada junto com a cláusula WHERE, que especifica exatamente a linha da tabela que terá seus dados alterados.

No exemplo acima, adicionamos um novo registro na tabela “funcionarios”, mas agora queremos atualizar um dos dados que inserimos, mais precisamente o cargo que foi especificado. Para isso, usamos o comando:

```
UPDATE funcionarios SET cargo = 'Analista de Sistemas e Aplicações' WHERE id = 1;
```

SQL DELETE

O comando DELETE, como você já deve imaginar, é usado para **excluir dados de uma tabela**. Assim como o anterior, ele também é usado junto com a cláusula WHERE. Nesse caso, vamos imaginar que não precisamos mais das informações que inserimos nos exemplos acima e queremos apagá-las da nossa tabela. Podemos fazer isso da seguinte forma:

```
DELETE FROM funcionarios WHERE id = 1;
```

SQL SELECT

O comando SELECT permite definir critérios para **realizar consultas aos registros que foram armazenados no banco de dados**. Nesse caso, a forma mais simples de busca que podemos fazer é consultar todos os campos de uma tabela, dessa forma:

```
SELECT id, nome, cargo FROM funcionarios;
```

Entretanto, existem diversas outras maneiras de realizar consultas e, utilizando os operadores corretos, podemos deixar nossas buscas mais refinadas.

Para ficar mais evidente, imagine que você quer consultar o nome do funcionário com id = 15. Novamente, para especificar exatamente qual é o dado que queremos visualizar, utilizamos a cláusula WHERE, conforme mostramos no comando abaixo:

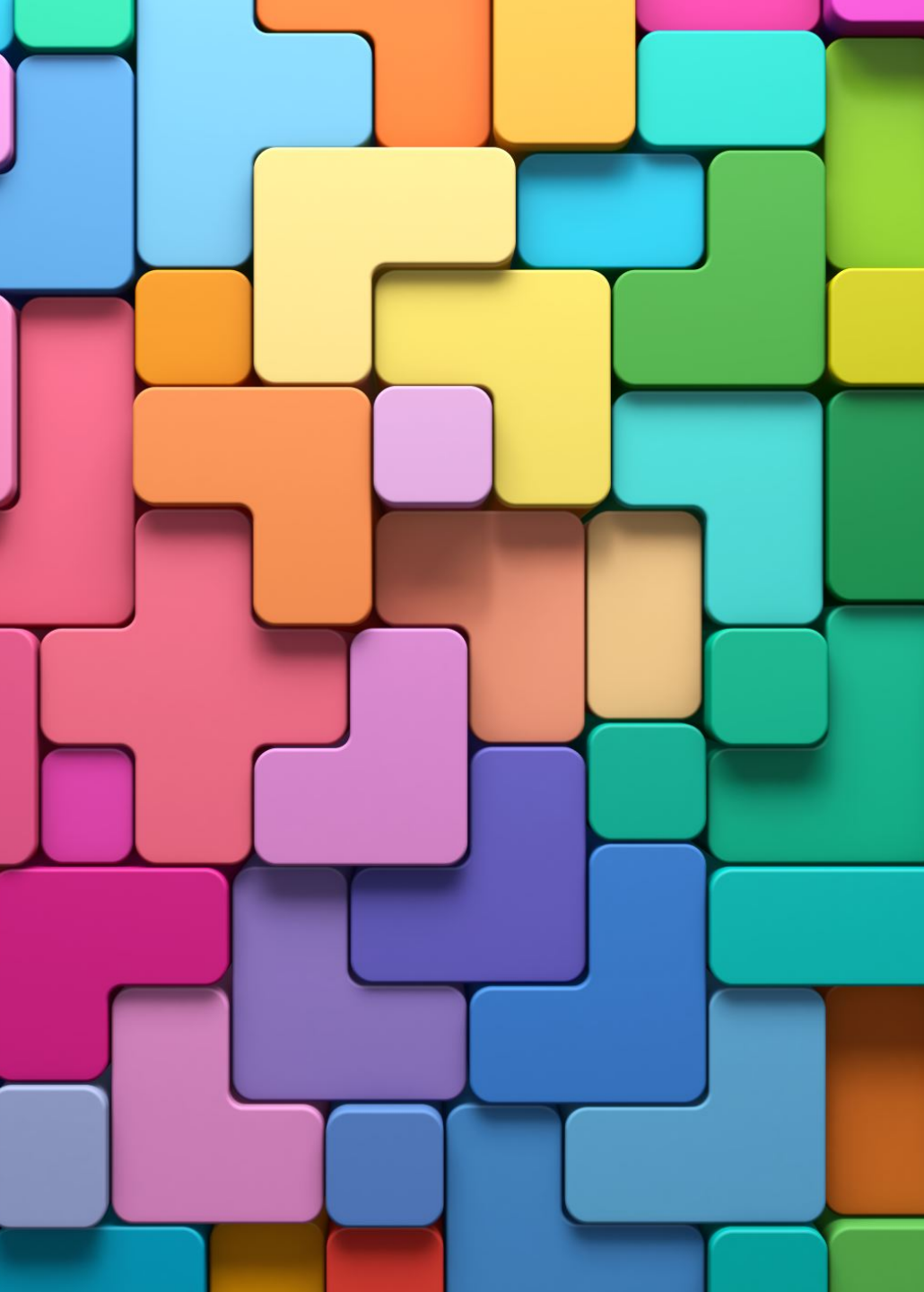
```
SELECT nome FROM funcionarios WHERE id = 15;
```

Unindo tabelas baseado na relação entre colunas: SQL JOIN

Como dissemos, muitas vezes é necessário fazer buscas mais complexas na base de dados e podemos utilizar diversas instruções para refinar nosso SELECT. Entre elas, estão as cláusulas JOIN.

Com elas, conseguimos **combinar linhas de várias tabelas com base na relação existente entre as colunas dessas tabelas**. Para ficar mais nítido, observe para qual caso cada uma delas é usada:

- **INNER JOIN:** retorna os registros que têm valores correspondentes em ambas as tabelas;
- **LEFT JOIN:** retorna todos os registros da tabela esquerda, além dos registros com valores correspondentes na tabela direita;
- **RIGHT JOIN:** retorna todos os registros da tabela direita, além dos registros com valores correspondentes na tabela esquerda;
- **FULL JOIN:** retorna todos os registros de ambas as tabelas quando há uma correspondência entre elas.



Referências Bibliográficas

FRANÇA, R. **SQL: O que é e como usar os principais comandos básicos SQL**. Disponível em: <https://blog.betrybe.com/sql/> . Acesso em: 12 mar. 2025.